

PSR-PIPELINE: A Reproducible sEMG Benchmarking Workflow for Finger Intent Prediction in Post-Stroke Rehabilitation

Repository-derived manuscript draft

July 11, 2026

Abstract

Reliable finger-intent decoding from surface electromyography (sEMG) is attractive for post-stroke rehabilitation, but it is difficult to study reproducibly when preprocessing choices, tensor layouts, and model evaluation code are scattered across separate scripts. This paper documents PSR-PIPELINE, a repository-centered workflow that packages a three-stage sEMG preprocessing stack, a twelve-feature window representation, shared model adapters, and benchmark utilities for multi-label finger-intent prediction on PHYSIOMIO [5, 1]. The current pipeline filters raw EMG with a 20–450 Hz zero-phase Butterworth stage, denoises it with wavelet soft-thresholding, segments 200 ms windows with 50% overlap, and converts each window into a channel-by-feature tensor that can be consumed by LSTM, CNN, and graph neural network predictors. On the repository’s held-out test artifacts, the LSTM achieves the strongest exact-match accuracy, finger accuracy, and macro AUROC (0.545, 0.784, and 0.858), while the GNN achieves the strongest macro F1 and AUPRC (0.706 and 0.776). These results position PSR-PIPELINE as a useful systems benchmark rather than a single-model state-of-the-art claim, because the available evidence still covers one dataset and does not yet include full ablation or cross-dataset generalization studies. Even with that limitation, the released pipeline, metrics, and paper-support artifacts establish a concrete base for future rehabilitation-oriented sEMG experiments.

1 Introduction

Finger-intent prediction from sEMG is a clinically meaningful interface problem because it can connect residual forearm muscle activity to assistive control, therapy feedback, and staged post-stroke motor retraining [8]. In practical terms, the task is to recover a five-finger activation pattern from multichannel forearm sEMG so that downstream rehabilitation software or hardware can react to a user’s intended movement.

The technical difficulty is not only model accuracy, but also pipeline reproducibility. Post-stroke sEMG is noisy, weakly structured, and sensitive to subject-specific recording conditions, so reported performance depends heavily on filtering, denoising, segmentation, label construction, and data splitting. Public repositories often expose only part of that path, which makes it hard to compare alternative predictors under a consistent signal-processing and evaluation protocol.

PSR-PIPELINE addresses that gap by organizing the full workflow in one codebase [5]. The repository combines preprocessing modules, feature extraction, tensor adapters, dataset loaders, three predictor families, and saved benchmark metrics. It also includes additional two-stage healthy-to-impaired training scripts and summary files, although those transfer-learning artifacts are less uniformly packaged than the main benchmark outputs and therefore are treated as secondary evidence in this manuscript.

The central story of this paper is therefore systems-oriented rather than novelty-driven. We do not claim a new decoding architecture. Instead, we document a reusable benchmark workflow that makes the following engineering choices explicit: a fixed preprocessing specification, a shared window-level representation for model comparison, patient-level data splitting to reduce leakage, and repository-local metric artifacts that can be audited after training.

This updated paper makes four contributions. First, it formalizes the current preprocessing and feature pipeline implemented in the repository. Second, it explains how the code standardizes inputs so that LSTM, CNN, and GNN models can be compared inside one training and evaluation framework. Third, it consolidates saved metrics from all three predictor families, including the repository’s existing LSTM artifacts that are not highlighted in the top-level README table. Fourth, it reports the present scope limits of the benchmark so that future work can extend the repository with clearer ablations, stronger transfer studies, and broader rehabilitation validation.

2 Related Work

sEMG has long been studied as a non-invasive control channel for rehabilitation, prosthetics, and human-machine interaction, with performance shaped by both acquisition quality and downstream decoding choices [8]. That literature makes clear that signal conditioning is not a minor implementation detail: denoising, spectral shaping, and feature design directly affect the reliability of any intent-decoding model.

Sequence-oriented deep models remain a natural fit for windowed biosignal decoding because they can aggregate temporal evidence across successive frames. In particular, LSTM-based models are well suited to sequential dependencies in noisy measurements [3], while compact 1D CNNs can emphasize local temporal motifs and efficient inference. The current repository follows both lines by exposing a recurrent baseline and several CNN size variants under one interface.

Graph neural networks provide a complementary viewpoint by modeling structured relationships among windowed observations rather than treating the sequence only as a flat timeline [4, 2, 6]. PSR-PIPELINE uses that idea to construct a graph over window-level nodes and then applies configurable graph operators before graph-level pooling. Relative to these prior model families, the distinguishing contribution here is not a new graph formulation or recurrent block, but a repository-level benchmark that keeps preprocessing, tensorization, and metric reporting consistent across heterogeneous predictors.

3 Method

3.1 Overview

PSR-PIPELINE implements a modular decoding workflow that starts from raw multichannel sEMG and ends with five sigmoid-compatible logits, one for each finger class. The method is organized around four modules: signal preprocessing, window-level feature extraction, tensor adaptation, and model-specific prediction heads. Figure 1 summarizes the repository pipeline and the shared interfaces between modules.

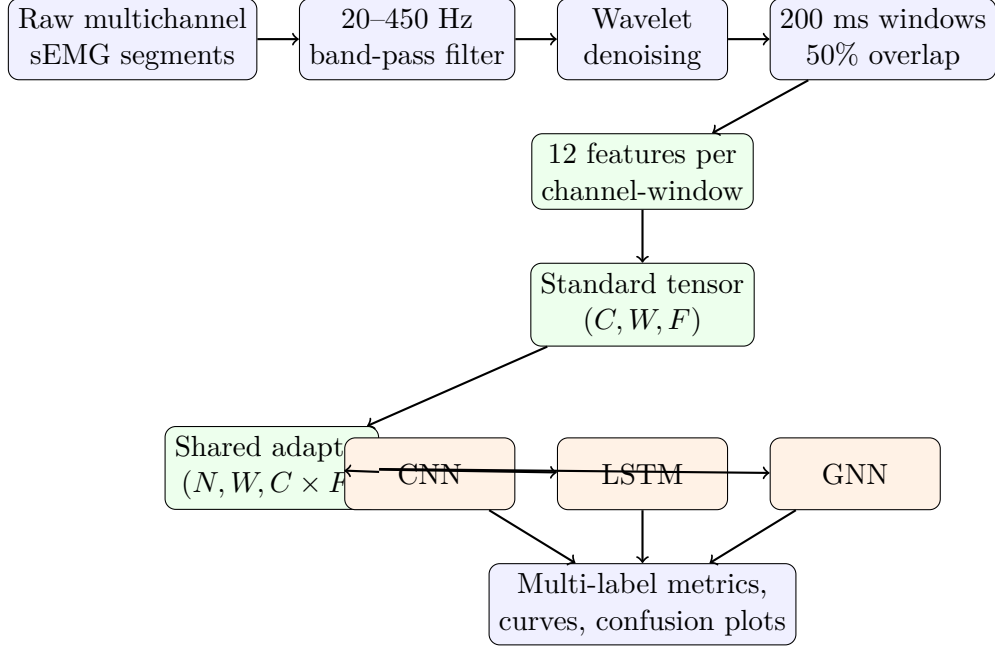


Figure 1: Repository pipeline documented in this paper. The same preprocessing and feature representation are reused across LSTM, CNN, and GNN predictors, which makes the saved benchmark artifacts directly comparable.

3.2 Signal preprocessing

The preprocessing module is explicitly parameterized by a frozen configuration object in the repository. Raw sEMG first passes through a fourth-order Butterworth band-pass filter with 20–450 Hz cutoffs and zero-phase forward-backward application. This stage suppresses low-frequency motion artifacts and high-frequency electrical noise while preserving time alignment across channels.

The filtered signal is then denoised with wavelet soft-thresholding. The default configuration uses a Symlet-4 basis, four decomposition levels, universal threshold selection, and a median-based noise estimate. Although the repository does not yet include an ablation isolating this choice, the implementation makes the denoising rule explicit and reproducible.

After denoising, the continuous recording is segmented into 200 ms windows with 50% overlap. These window settings balance temporal responsiveness with a sufficient sample count for stable summary statistics. The repository disables zero-padding by default, so each window represents observed signal rather than synthetic boundary content.

3.3 Window-level feature representation

Each channel-window pair is summarized by twelve handcrafted descriptors spanning time and frequency domains. The time-domain set includes RMS, mean absolute value, integrated EMG, waveform length, variance, zero crossings, slope sign changes, and Willison amplitude. The frequency-domain set includes mean frequency, median frequency, spectral entropy, and total power, with spectral quantities estimated from Welch periodograms [7]. The resulting tensor has shape (C, W, F) , where C is the number of channels, W is the number of windows, and $F = 12$ is the feature count.

3.4 Shared tensor adapter

The repository standardizes cross-model comparison with one shared adapter. It transforms (C, W, F) tensors into $(N, W, C \times F)$ sequences, where windows act as time steps and all channel features are concatenated into one vector per step. This small interface decision matters because it allows recurrent, convolutional, and graph-based predictors to consume the same derived representation without duplicating preprocessing code.

3.5 Predictor families

The LSTM branch applies two stacked recurrent layers with hidden sizes 128 and 256, followed by a 128-unit fully connected layer and dropout before five output logits. This branch emphasizes temporal accumulation across windows and, in the saved metrics, yields the strongest exact-match accuracy, finger accuracy, and AUROC.

The CNN branch wraps several 1D convolutional student models behind a channels-first permutation layer. In the current training code, the benchmarked CNN consumes the shared window-level sequence after permutation from $(N, W, C \times F)$ to $(N, C \times F, W)$. The repository supports multiple model sizes, but the saved benchmark artifacts are presented as one CNN baseline rather than a full architecture sweep.

The GNN branch interprets each time window as a node in a graph. The implementation supports GCN, GraphSAGE, and GAT operators [4, 2, 6]; the published repository narrative highlights a two-layer, 64-hidden-dimension configuration with mean pooling. In the current code, graphs are built densely across windows within each sample, enabling message passing over the full temporal context before graph-level pooling and readout.

3.6 Training and evaluation protocol

The training utilities unify optimization and reporting across model families. The repository uses binary cross-entropy with logits for five-label prediction, Adam-based optimization, plateau-triggered learning-rate reduction, checkpointing, and saved metric curves. Evaluation reports exact-match accuracy, per-finger averaged accuracy, macro precision, macro recall, macro F1, macro AUROC, and macro AUPRC. The metrics module also stores per-finger scores and task-level curve data, which makes both aggregate and finger-wise analysis available after training.

4 Experiments

4.1 Dataset and split construction

All benchmark artifacts analyzed in this paper come from the repository’s PHYSIOMIO processing path [1, 5]. The data loader searches raw parquet files, extracts contiguous movement segments, maps gesture labels to five-bit finger targets, preprocesses each segment, and stores processed tensors as PyTorch split files.

The split logic is patient-level rather than segment-level. Patients are shuffled with a fixed seed and divided into 70% train, 10% validation, and 20% test partitions, which reduces identity leakage between train and test segments. The default loader configuration targets impaired-arm recordings

Table 1: Held-out test performance from the repository metric files. Bold numbers mark the best score in each column.

Model	Exact-match	Finger Acc.	Macro F1	Macro AUROC	Macro AUPRC
LSTM	0.545	0.784	0.705	0.858	0.754
GNN	0.448	0.683	0.706	0.787	0.776
CNN	0.442	0.694	0.676	0.767	0.764

unless the user explicitly changes the arm-split setting, so the main benchmark should be interpreted as a post-stroke impaired-arm decoding study.

4.2 Implementation scope

The repository exposes a richer experimental surface than the evidence-complete benchmark summarized in this paper. In addition to direct LSTM, CNN, and GNN runs, the codebase contains healthy-to-impaired pretraining and finetuning scripts, optional threshold tuning, class-imbalance weighting, and multiple graph and CNN variants. However, not all of those branches are reported with equally complete comparison tables, so this paper centers its headline claims on the metric files already saved under ‘metrics/’.

4.3 Evaluation protocol

The README defines the benchmark as five-class multi-label classification with a nominal decision threshold of 0.5 on the held-out test split. For transparency, the metric implementation also records exact-match accuracy and per-finger summaries, which is important because different models may trade precision for recall while achieving similar binary correctness at the finger level.

This paper follows the experimental writing guidance of the external ‘Research-Paper-Writing-Skills’ workflow by tying every headline claim to an existing artifact. Because no ablation package is currently stored in the repository, we deliberately avoid module-level causality claims and instead focus on benchmark-level observations that are directly supported by saved outputs.

5 Results

The current repository benchmark exposes a clear trade-off between balanced sequence modeling and high-recall graph modeling. Table 1 shows that the LSTM achieves the strongest exact-match accuracy, finger accuracy, and macro AUROC, while the GNN narrowly leads on macro F1 and macro AUPRC. The CNN remains competitive as a lightweight baseline but does not surpass the best LSTM or GNN score on any headline metric in the saved artifacts.

The gap between the GNN and the LSTM is easiest to interpret through precision-recall behavior. The GNN posts the highest macro recall among the saved runs (0.832 versus 0.768 for the LSTM and 0.697 for the CNN), which lifts its macro F1 and AUPRC. The LSTM, however, converts that lower recall into materially stronger exact-match behavior and substantially stronger AUROC, suggesting a more balanced discrimination profile across thresholds.

Finger-wise scores show that the ranking is not uniform across all outputs. Table 2 indicates that the

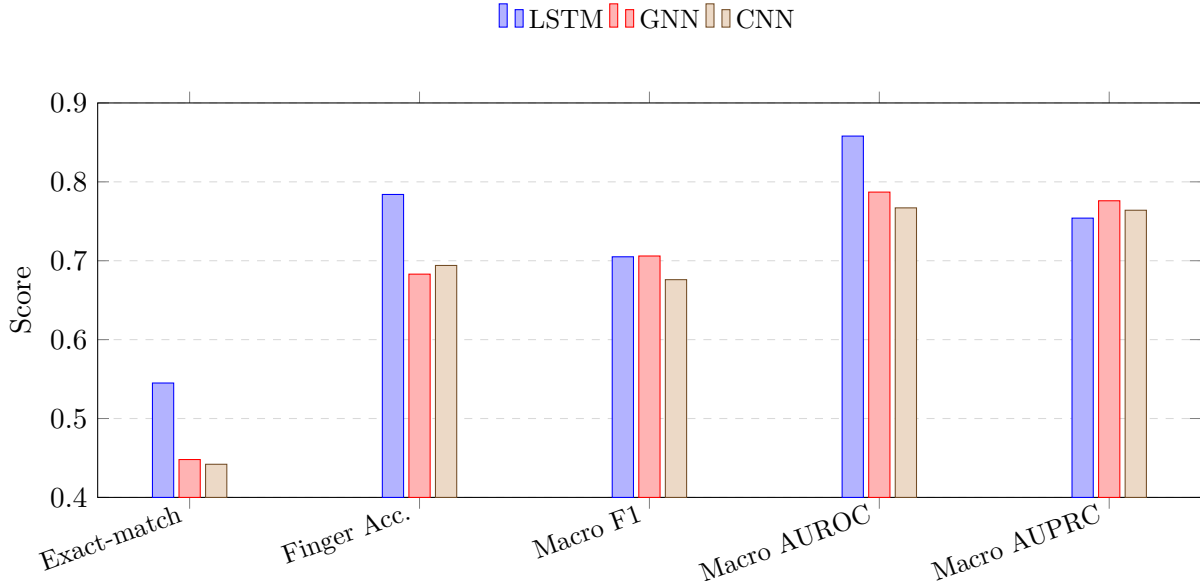


Figure 2: Metric-level comparison across the three saved model families. The GNN leads on recall-sensitive summary metrics, while the LSTM leads on exact-match accuracy, finger accuracy, and macro AUROC.

Table 2: Per-finger F1 score by model family. Bold numbers mark the strongest model for each finger.

Finger	LSTM	GNN	CNN
Thumb	0.716	0.776	0.726
Index	0.723	0.696	0.660
Middle	0.712	0.689	0.675
Ring	0.696	0.688	0.670
Little	0.680	0.678	0.649

GNN is strongest on the thumb label, while the LSTM is strongest on the index, middle, ring, and little finger labels. This pattern is consistent with the repository-wide picture: graph aggregation appears to favor aggressive positive detection, whereas the recurrent model distributes performance more evenly across fingers.

One additional paper-level update is worth noting. The top-level repository README foregrounds the GNN and CNN comparison, but the repository also contains complete LSTM metric artifacts under ‘metrics/lstm/'. Including those saved LSTM results materially changes the interpretation of the benchmark, because the best overall system is no longer a simple GNN-versus-CNN choice.

6 Discussion

The strongest conclusion supported by the present evidence is that PSR-PIPELINE is already valuable as a reproducible benchmarking scaffold even though it is not yet a submission-ready algorithm paper. The preprocessing code, feature representation, and metric artifacts are detailed

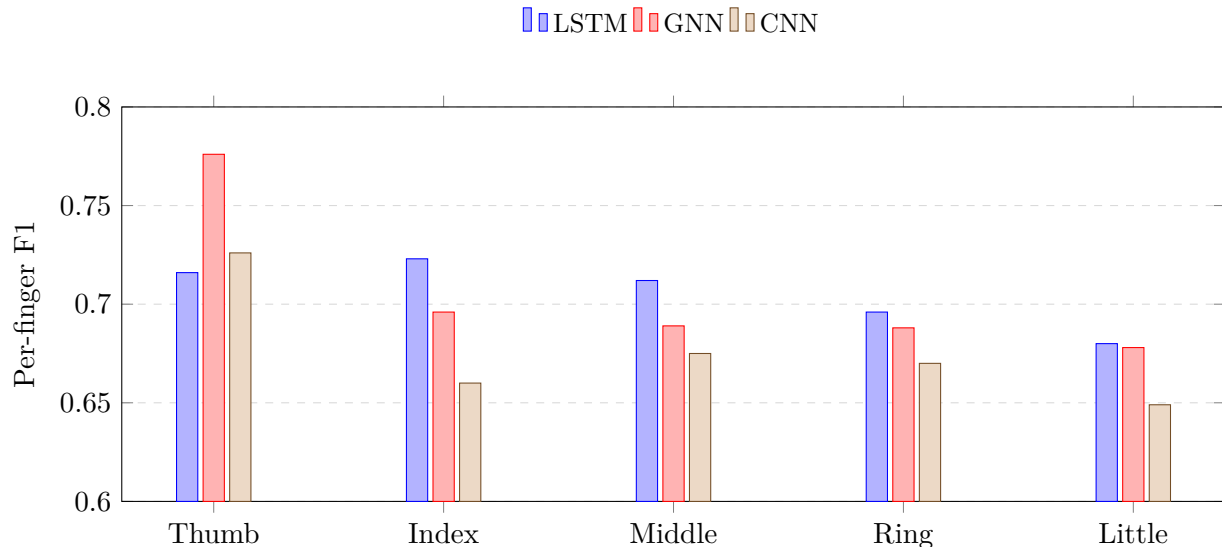


Figure 3: Per-finger F1 comparison derived from the saved metric files. The LSTM wins four of five finger labels, while the GNN is strongest on the thumb class.

enough for another engineer or researcher to rerun the benchmark logic, inspect trade-offs, and extend the model suite without reconstructing the full pipeline from scratch.

The benchmark itself reveals two useful modeling lessons. First, a straightforward recurrent model is hard to dismiss in this setting: the saved LSTM run provides the best exact-match decoding quality and the strongest macro AUROC, which are attractive traits for downstream control. Second, the graph branch is not redundant, because its higher recall and stronger macro AUPRC suggest that explicit cross-window message passing may recover positive finger activations that the other models miss.

The repository also hints at a broader transfer-learning direction. The `run_two_stage.py` script and `results/README.md` file summarize healthy-to-impaired pretraining and finetuning results for CNN and LSTM branches. Those artifacts are promising, especially because rehabilitation deployments often face limited impaired-arm data, but they are packaged differently from the main benchmark tables. For that reason, this paper mentions them as follow-on evidence rather than merging them into the headline comparison.

Several limitations remain. The benchmark currently covers one dataset, one main processed benchmark path, and no stored ablation study for preprocessing or architecture variants. The GNN implementation uses dense window graphs, which may become expensive as sequence length grows. The repository also includes a Mamba notebook for signal translation, but that component is not integrated into the same evidence-complete evaluation path as the direct finger-intent predictors. These gaps do not invalidate the current benchmark, but they do define the most important next steps for turning PSR-PIPELINE from a strong engineering base into a stronger empirical paper.

7 Conclusion

This paper documents PSR-PIPELINE as a reproducible workflow for sEMG-based finger-intent prediction in a post-stroke rehabilitation setting. The key idea is not a new standalone network,

but a repository structure that makes preprocessing, feature extraction, tensor adaptation, model comparison, and metric reporting explicit and reusable.

The strongest current evidence favors the LSTM for exact-match accuracy, finger accuracy, and macro AUROC, while the GNN offers the best macro F1 and AUPRC. That trade-off makes the repository useful for controlled model comparison rather than single-number leaderboard claims. A current limitation is evaluation scope: the benchmark is still centered on one dataset and lacks stored ablations for several design choices. Extending the same workflow to broader baselines, richer transfer settings, and more clinically realistic validation is the clearest next step.

References

- [1] Formove AI. PhysioMio. <https://huggingface.co/datasets/formove-ai/physiomio>, 2026. Dataset page, accessed July 11, 2026.
- [2] William L. Hamilton, Rex Ying, and Jure Leskovec. Inductive representation learning on large graphs. In *Advances in Neural Information Processing Systems*, 2017.
- [3] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8): 1735–1780, 1997.
- [4] Thomas N. Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. In *International Conference on Learning Representations*, 2017.
- [5] Post-Stroke Rehabilitation Project. PSR-Pipeline. <https://github.com/post-stroke-rehab/psr-pipeline>, 2026. GitHub repository, accessed July 11, 2026.
- [6] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. Graph attention networks. In *International Conference on Learning Representations*, 2018.
- [7] Peter D. Welch. The use of fast fourier transform for the estimation of power spectra: A method based on time averaging over short, modified periodograms. *IEEE Transactions on Audio and Electroacoustics*, 15(2):70–73, 1967.
- [8] Mingde Zheng, Michael S. Crouch, and Michael S. Eggleston. Surface electromyography as a natural human-machine interface: A review. *arXiv preprint arXiv:2101.04658*, 2021.